TD11: Correction

CPES ≪ Sciences des données, arts et cultures ≫

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

18 mai 2026

Exercice 1

Question 1

On cherche à montrer q'un Arbre Binaire Complet (ABC) a au plus 2^k sommets de profondeur k. On a alors la garantie que la hauteur de l'ABC est d'au moins k.

Ceci implique donc que les niveaux $\{0, \dots, k-1\}$ de l'ABC sont tous *complets*. Cela correspond pour le niveau 0 à n'avoir qu'un sommet (la raçine r), puis pour le niveau $2^0 = 1$ à avoir les deux enfants de r ($2 \times 2^0 = 2^1 = 2$), puis au niveau 2 à avoir les deux enfants des deux enfants de r ($2 \times 2^1 = 2^2 = 4$), et ainsi de suite. A un niveau intermédiaire $i \in \{0, \dots, k-1\}$, on a donc exactement $2 \times 2^{i-1} = 2^i$ sommets à ce niveau. Au niveau k, on a alors bien *au maximum* $2 \times 2^{k-1} = 2^k$ sommets, car le niveau k est éventuellement le dernier, et peut donc être incomplet.

Question 2

L'arbre est de hauteur h : les $(h-1)$ premiers niveaux sont assurément tous remplis (avec exactement 2^i sommets, pour tout $i \in \{0, \dots, h-1\}$). La variation sur le nombre exact de sommets de l'arbre réside sur le dernier niveau, qui peut avoir entre 1 et 2^h sommets (résultat de la **Question 1**). Nous allons donc traiter les deux cas, afin d'encadrer le nombre de sommets n par $n_{\min}$ (le cas où le niveau h n'a qu'un sommet), et $n_{\max}$ (le cas où le niveau h est complet, avec 2^h sommets).

$\underline{n_{\min}}$ Si le dernier niveau h ne contient qu'un sommet, on a :

$$
\begin{aligned}
n_{\min} &= 2^0 + 2^1 + \cdots + 2^{h-1} + 1 \\
&= \left(\sum_{i=0}^{h-1} 2^i \right) + 1 \\
&= 1 + \sum_{i=0}^{h-1} 2^i \\
&= 1 + \frac{1 - 2^{(h-1)+1}}{1-2} \\
&= 1 + \frac{2^h - 1}{1} \\
&= 1 + 2^h - 1 \\
&= 2^h
\end{aligned}
$$

La simplification de la somme est faite avec le calcul d'une *série géométrique*, qui, $\forall q > 1, \forall k \in \mathbb{N}$, donne :

$$
1 + q + \cdots + q^k = \sum_{i=0}^{k} q^i = \frac{1 - q^{k+1}}{1 - q}
$$

$\underline{n_{\textbf{max}}}$ On applique le même raisonnement en remplaçant 1 par 2^h :

$$
\begin{aligned}
n_{\min} &= 2^0 + 2^1 + \cdots + 2^{h-1} + 2^h \\
&= \sum_{i=0}^{h} 2^i \\
&= \frac{1 - 2^{h+1}}{1-2} \\
&= \frac{2^{h+1} - 1}{1} \\
&= 2^{h+1} - 1
\end{aligned}
$$

Ainsi, on a bien $n \in [n_{\min}, n_{\max}] = \left[2^h, 2^{h+1} - 1\right]$.

Question 3

On cherche à exprimer la hauteur d'un ABC à partir du nombre de sommets n. On peut repartir du résultat de la **Question 2** :

$$
\begin{aligned}
& 2^h \leqslant n \leqslant 2^{h+1} - 1 \\
\Leftrightarrow & 2^h \leqslant n < 2^{h+1} \\
\Leftrightarrow & \log_2\left(2^h\right) \leqslant \log_2(n) < \log_2\left(2^{h+1}\right) \\
\Leftrightarrow & h \leqslant \log_2(n) < h+1 \\
\Leftrightarrow & h = \lfloor \log_2(n) \rfloor
\end{aligned}
$$

Question 4

Considérons un ABC de hauteur $h \geqslant k$ (condition nécessaire pour avoir des sommets de hauteur k). Pour rappel, un sommet est de hauteur k ssi il existe une feuille distante de k arêtes.

Si l'ABC est complètement rempli au dernier niveau, alors tous les sommets de hauteur k seront au niveau atteint en partant du niveau h et en *remontant* k fois l'arête parente, soit au niveau $(h-k)$.

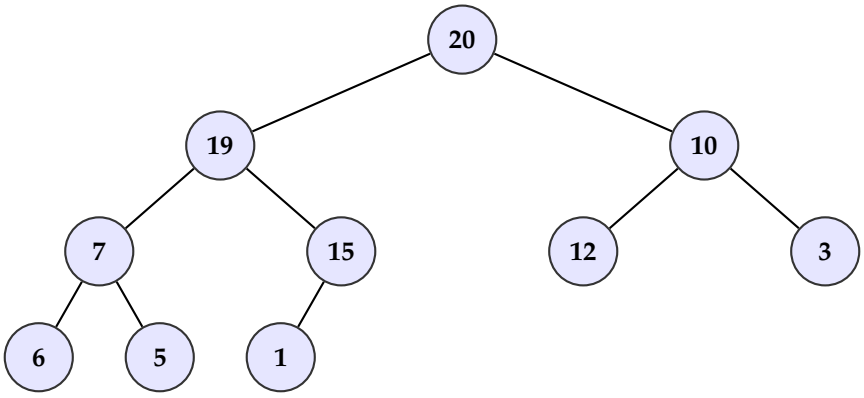
Or le nmbre de sommets à ce niveau est 2^{h-k}, qui constitue bien la borne supérieure sur le nombre de sommets de hauteur k d'un ABC de hauteur h. En effet, si le niveau h de l'arbre n'est pas complet, alors il faudra pour certains des sommets remonter d'un niveau de plus, sur lequel il y aura nécessairement moins de sommets que le niveau $(h-k)$.

Question 5

En représentant un ABC dans un tableau T, on ajoute les sommets dans T niveau par niveau. Or, on double le nombre de sommets à chaque niveau (sauf éventuellement sur le dernier) : ainsi, pour trouver leurs enfants, et **en indexant le tableau T à partir de** 0, on accèdera à l'indice $2i+1$ pour le fils gauche, et $2i+2$ pour le fils droit. Si le sommet i est une feuille, alors il n'a pas d'enfant, et on oberservera $2i+2 > 2i+1 > |T|$.

Exercice 2

Le tableau $[20, 19, 10, 7, 15, 12, 3, 6, 5, 1]$ est représenté en ABC en insérant les éléments de gauche à droite, niveau par niveau (indexation à partir de 0 : le nœud i a pour fils gauche $2i+1$ et fils droit $2i+2$, voir **Question 5, Exercice 1**).



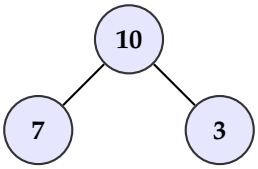
Exercice 3

Question 1

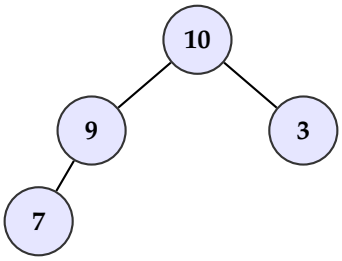
On insère les éléments $10, 7, 3, 9, 11, 5, 6, 4, 8$ un à un. Chaque insertion place le nouvel élément à la fin du tas, puis remonte (AUGMENTER-CLÉ) tant que le père est plus petit. On

Étapes de construction :

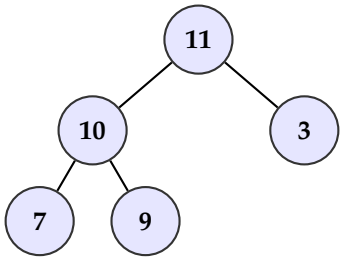
Après insertion de 10, 7, 3 : 7 et 3 sont respectivement fils gauche et droit de 10, aux positions 1 et 2 (la raçine est en position 0). Aucune remontée nécessaire.



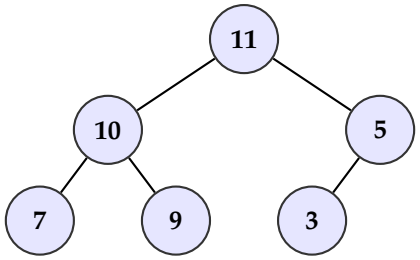
Insertion de 9 : il entre en position 3 (fils gauche de 7). $9 > 7 \Rightarrow$ échange $9 \leftrightarrow 7$. $9 < 10 \Rightarrow$ stop.



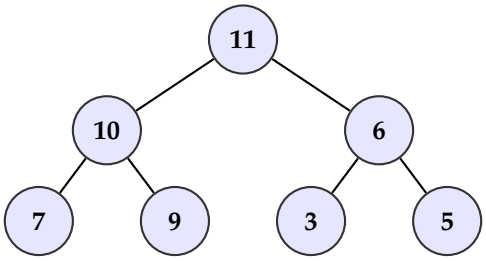
Insertion de 11 : position 3 (fils droit de 9). $11 > 9 \Rightarrow$ échange $11 \leftrightarrow 9$. $11 > 10 \Rightarrow$ échange $11 \leftrightarrow 10$.



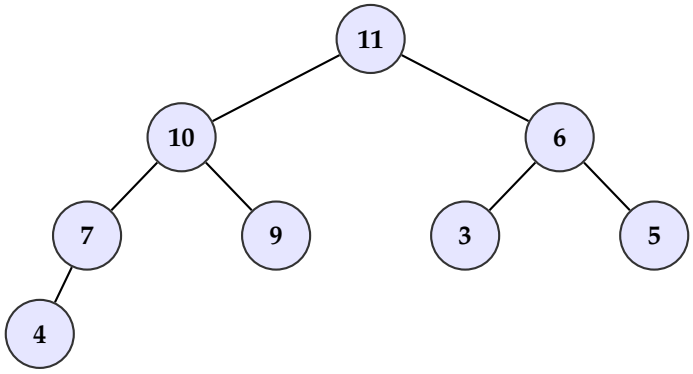
Insertion de 5 : position 5 (fils gauche de 3). $5 > 3 \Rightarrow$ échange $5 \leftrightarrow 3$. $5 < 11 \Rightarrow$ stop.



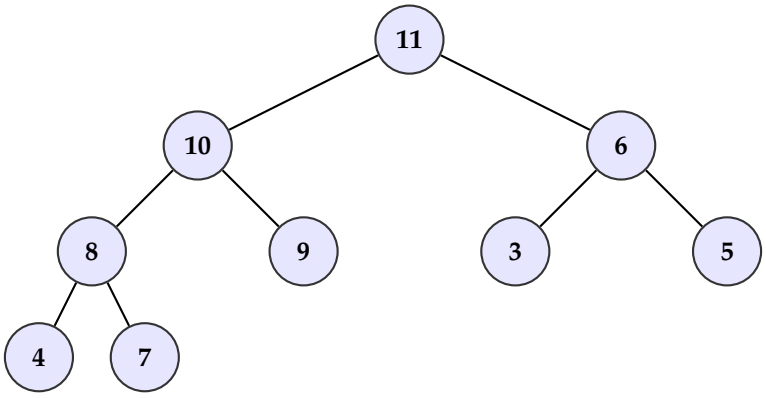
Insertion de 6 : position 6 (fils droit de 5). $6 > 5 \Rightarrow$ échange $6 \leftrightarrow 5$. $6 < 11 \Rightarrow$ stop.



Insertion de 4 : position 7 (fils gauche de 7). $4 < 7 \Rightarrow$ stop. Aucun échange.



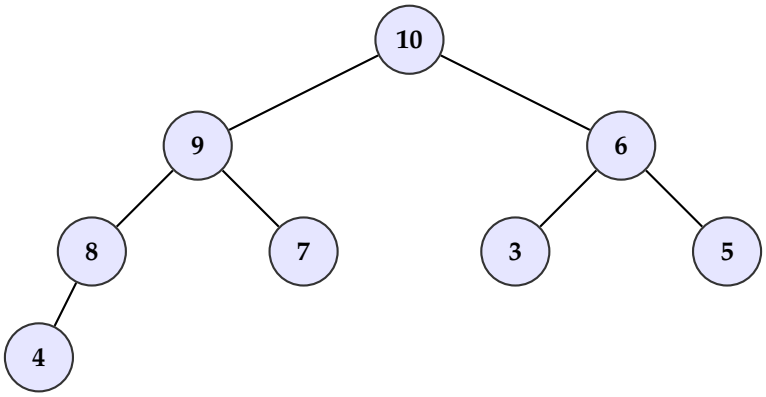
Insertion de 8 : position 8 (fils droit de 7). $8 > 7 \Rightarrow$ échange $8 \leftrightarrow 7$. $8 < 10 \Rightarrow$ stop.



Question 2

On retire **11**, qu'on remplace par 7 à la racine (dernier élément du tas).
Sommet courant $= 7$ (à la raçine), fils : 10 et 6. max $= 10 > 7 \Rightarrow$ échange $7 \leftrightarrow 10$.
Sommet courant $= 7$ (position 1), fils : 8 et 9. max $= 9 > 7 \Rightarrow$ échange $7 \leftrightarrow 9$.
Sommet 7 en position 4 : aucun fils $\Rightarrow$ stop.

Nouveau tas-max après extraction de 11 :



Exercice 4

Dans une *File de Priorité* (FP), c'est la *clé* qui détermine la priorité des sommets. Comme expliqué dans le cours, on peut construire deux types de FP : les FP « min » (qui retourne l'élément minimal en $O(1)$ et rééquilibre le tas en $O(log(n))$) et les FP « max » (qui retourne l'élément maximal en $O(1)$ et rééquilibre le tas en $O(log(n))$).

D'un autre côté, que l'on s'intéresse aux *piles* ou aux *files*, on doit gérer des *flux* d'éléments qui arrivent dans un certain *ordre*. On utilise $\mathbb{N}$ pour associer à ces éléments une clé par ordre croissant d'arrivée (le premier élément a pour clé 0, le second 1, etc.).

Pour les files, on opère en *FIFO* : premier arrivé, premier sorti. On veut donc faire sortir en premier les éléments les plus vieux, et donc dont les clés sont les plus petites :

on va donc vouloir retourner l'élément de clé minimale à chaque itération, on utilisera donc une FP « min ».

Pour les piles, on opère en *LIFO* : dernier arrivé, premier sorti. On veut donc faire sortir en premier les éléments les plus récents, et donc dont les clés sont les plus grandes : on va donc vouloir retourner l'élément de clé maximale à chaque itération, on utilisera donc une FP « max ».

Notez qu'on pourrait aussi décider de fixer l'un des deux types de FP (min ou max), et ensuite adapter la fonction d'assignation de clés pour implémenter une file ou une pile. Par exemple, si l'on fixe le choix d'une FP « min »et que l'on veut implémenter une pile, alors il suffit d'associer des clés négatives décroissantes par ordre d'arrivée (0 pour le premier élément, -1 pour le second, etc.).

Exercice 5

Voir la correction du livre de référence : Exercice 9, Chapitre 6.5, Cormen.